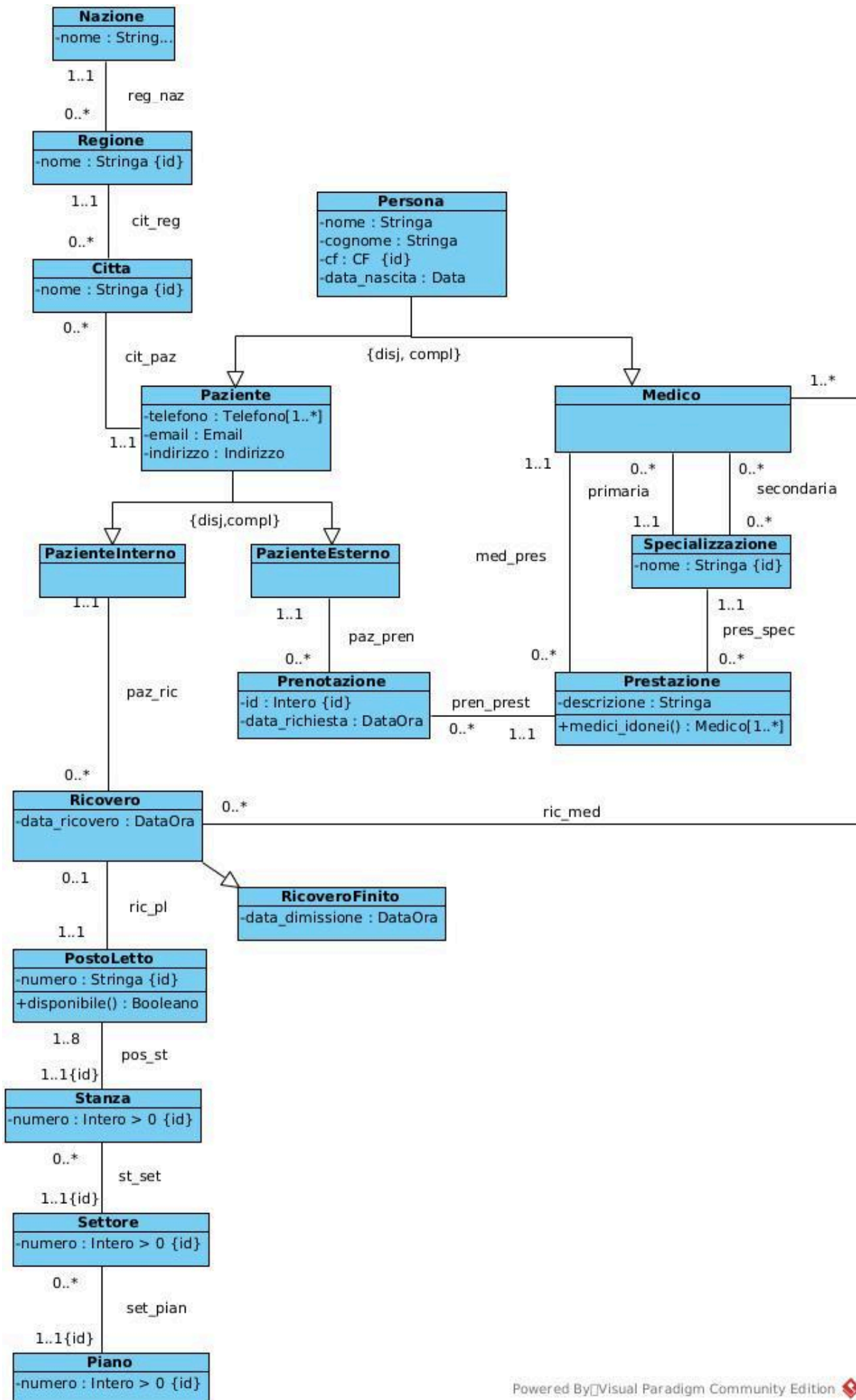


QuickHospital

1. Diagramma UML delle classi	2
1.1. Specifiche di Tipi di Dato	3
1.2. Specifiche delle classi	3
Vincoli Esterni	3
[V.RicoveroFinito.NonSovraposti]	3
[V.Ricovero.NonSovrapostoRicoveroFinito]	3
[V.RicoveroDopoDimissioni]	3
[V.RicoveroDopoVerificaDisponibilitaPostoLetto]	3
[V.RicoveroFinito.DimissioniDopoRicovero]	4
[V.Medico:SpecializzazionePrimariaDiversaDaSecondaria]	4
[V.Prestazione.EsistenzaMedicoConSpecializzazioneGiusta]	4
Operazioni ausiliarie	4
non_sovraposti_full(	4
non_sovraposti_half(	4
Specifica della classe PostoLetto	5
disponibile(): Booleano	5
Specifica della classe Prestazione	5
medici_idonei(): Medico[1..*]	5
2. Diagramma degli Use-Case	6
2.1. Specifiche di Use-Case Calcolare itinerario di visite	6
calcolo_percorso(m: Medico) : (Piano, Settore)[1..*]	6
registrazione_medico(	7

1.Diagramma UML delle classi



1.1. Specifiche di Tipi di Dato

Tipo Telefono: Stringa di 10 caratteri numerici secondo lo standart

Tipo Email: Stringa secondo lo standart

Tipo Indirizzo: tipo di record con campi

via: Stringa

civico: Stringa

cap: Intero > 0

Tipo CF: Stringa secondo lo standart

1.2. Specifiche delle classi

Vincoli Esterni

[V.RicoveroFinito.NonSovraposti]

Per ogni pi: PazienteInterno tale che (pi, rf1): paz_ric e (pi, rf2): paz ric, rf1: RicoveroFinito e rf2: RicoveroFinito e e rf1 ≠ rf2

deve essere vero che

- non_sovraposti_full(
rf1.data_ricovero,
rf1.data_dimissione,
rf2.data_ricovero,
rf2.data_dimissione) = True

[V.Ricovero.NonSovrapostoRicoveroFinito]

Per ogni pi: PazienteInterno tale che (pi, rf1): paz_ric e (pi, r2): paz ric, rf1: RicoveroFinito e r2: Ricovero deve essere vero che

- non_sovraposti_half(
rf1.data_ricovero,
rf1.data_dimissione,
r2.data_ricovero) = True

[V.RicoveroDopoDimissioni]

Per ogni pi: PazienteInterno tale che (pi, r1): paz_ric e (pi, r2): paz ric, r1: Ricovero e r2: Ricovero e r1 ≠ r2 deve essere vero che

- r1: RicoveroFinito
oppure
- r2: RicoveroFinito

[V.RicoveroDopoVerificaDisponibilitaPostoLetto]

Per ogni p: PazienteInterno

se e solo se esiste pl: PostoLetto tale che pl.disponibile() = True

viene creato r: Ricovero tale che

- r.data_ricovero è valorizzato e (r, pl): ric_pl

[V.RicoveroFinito.DimissioniDopoRicovero]

Per ogni rf: RicoveroFinito deve essere vero

- rf.data_ricovero < rf.data_dimissione

[V.Medico.SpecializzazionePrimariaDiversaDaSecondaria]

Per ogni m: Medico, sp1: Specializzazione, sp2: Specializzazione tali che (m, sp1): primaria e (m, sp2): secondaria deve essere vero che $sp1 \neq sp2$

[V.Prestazione.EsistenzaMedicoConSpecializzazioneGiusta]

Per ogni pr: Prestazione tale che (pr, sp): pres_spec. sp: Specializzazione deve esistere m: Medico tale che (m, sp): primaria oppure (m, sp): secondaria

[V.Prestazione.MedicoConSpecializzazioneGiusta]

Per ogni pr: Prestazione tale che (pr, m): med_pres, m: Medico e (pr, sp): pres_spec. sp: Specializzazione deve esistere (m, sp): primaria oppure (m, sp): secondaria

Operazioni ausiliarie

non_sovraposti_full(

d_in1: DataOra,
d_fin1: DataOra,
d_in2: DataOra,
d_fin2: DataOra
) : Booleano

precondizioni:

- d_in1 < d_fin1 e d_in2 < d_fin2

postcondizioni:

- operazione non fa side-effect
 - viene restituito valore "result" definito come segue:
 - se e solo se
 - d_fin1 < d_in2
oppure
 - d_fin2 < d_in1
- return True
- altrimenti
- return False

non_sovraposti_half(

d_in1: DataOra,
d_fin1: DataOra,
d_in2: DataOra
) : Booleano

precondizioni:

- d_in1 < d_fin1

postcondizioni:

- operazione non fa side-effect
- viene restituito valore “result” definito come segue:
 - se e solo se
 - $d_{fin1} < d_{in2}$
 return True
 - altrimenti
 - return False

Specifica della classe PostoLetto

disponibile(): Booleano

precondizioni: nessuna

postcondizioni:

- operazione non fa side-effect
- viene restituito il valore “result” definito come segue
 - se esiste r : Ricovero tale che $(r, this): ric_pl$ e r non è istanza di $RlcoveroFinito$
 - result = False
 altrimenti
 - result = True

Specifica della classe Prestazione

medici_idonei(): Medico[1..*]

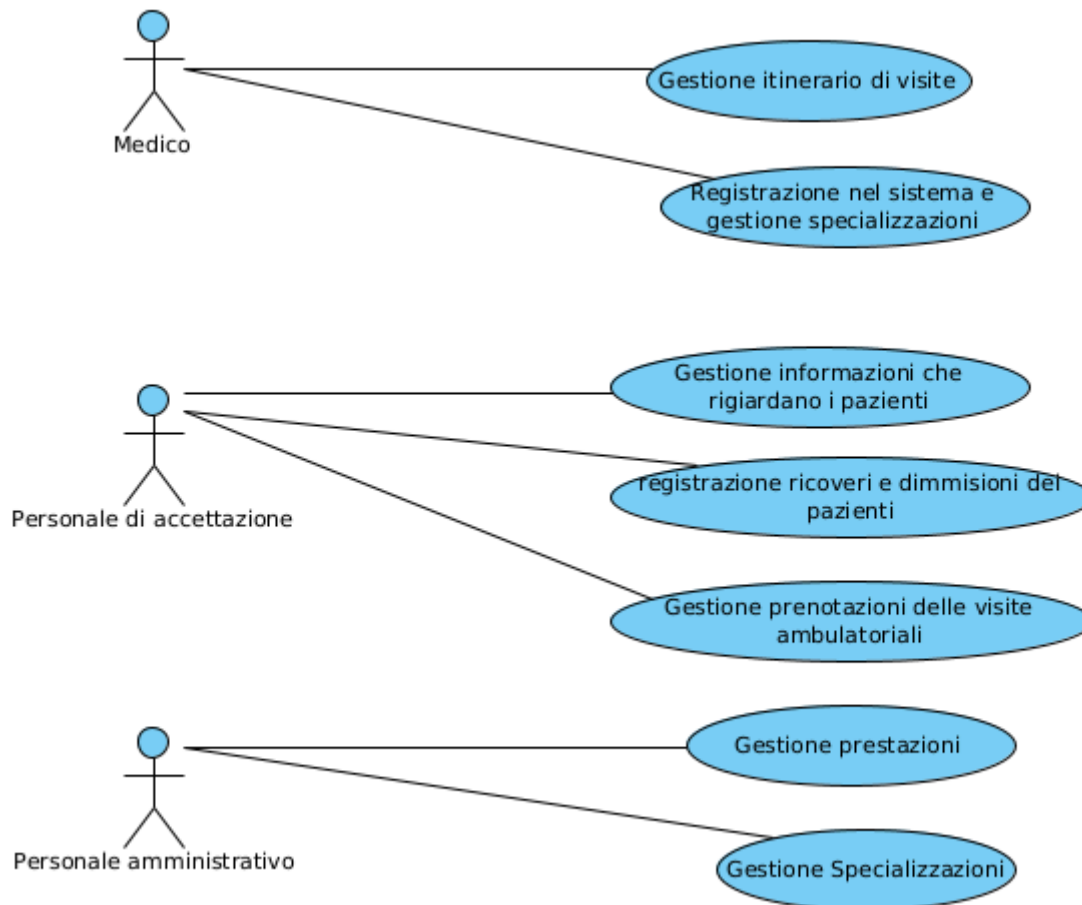
precondizione:

- esiste almeno un m : Medico tale che
 - $(this, m): med_pres$,
 - $(this, sp): pres_spec$, sp : Specializzazione
 - $(sp, m): primaria$
oppure
 - $(sp, m): secondaria$

postcondizioni:

- operazione non fa side-effect
- viene restituito il valore “result” definito come segue:
 - sia sp : Specializzazione tale che $(this, sp): pres_spec$
 - sia $M1$ insieme dei m : Medici tali che $(sp, m): primaria$
 - sia $M2$ insieme dei m : Medici tali che $(sp, m): secondaria$
 - se $|M1| > 0$
 - result = M1
 altrimenti
 - result = M2

2. Diagramma degli Use-Case



2.1. Specifiche di Use-Case Gestione itinerario di visite

$\text{calcolo_percorso}(m: \text{Medico}) : (\text{Piano}, \text{Settore})[1..*]$

precondizioni:

- esiste almeno un link $(m, r): \text{ric_med}$, $r: \text{Ricovero}$ e r non è istanza di RicoveroFinito

postcondizioni:

- operazione non fa side-effect
- viene restituito il valore "result" definito come segue:
 - sia R insieme di tutti $r: \text{Ricovero}$ tali che $(r, m): \text{ric_med}$ e r non è istanza di RicoveroFinito
 - per ogni r in R
 - sia $pl: \text{PostoLetto}$ tale che $(r, pl): \text{ric_pl}$
 - sia $st: \text{Stanza}$ tale che $(pl, st): \text{pos_st}$
 - sia $set: \text{Settore}$ tale che $(st, set): \text{st_set}$
 - sia $pian: \text{Piano}$ tale che $(set, pian): \text{set_pian}$
 - sia n la coppia di valori $(pian, set)$
 - sia P insieme di coppie n di ogni r in R ordinati in tale modo
 - per ogni coppie $(pian1, set1)$ e $(pian2, set2)$ deve essere vero che
 - $pian1 \leq pian2$
 - se $pian1 = pian2$ allora
 - $set1 < set2$
 - result P

2.2. Specifiche di Use-Case Registrazione nel sistema e gestione specializzazioni

registrazione_medico(

nom: Stringa,
cogn: Stringa,
codfisc: CF,
dn: Data,
sp_prim: Specializzazione,
SP: Specializzazione[0..*]
) : Medico

precondizione:

- non deve esistere un m: Medico tale che m.cf =codfisc
- sp_prim non è nel insieme SP

postcondizioni

- viene creato e restituito nuovo ogetto m: Medico con i valori:
 - m.nome = nom
 - m.cognome = cogn
 - m.cf = codfisc
 - m.data_nascita = dn
- viene creato il link (m, sp_prim): primaria
- per ogni sp in SP
 - viene creato il link (m, sp): secondaria